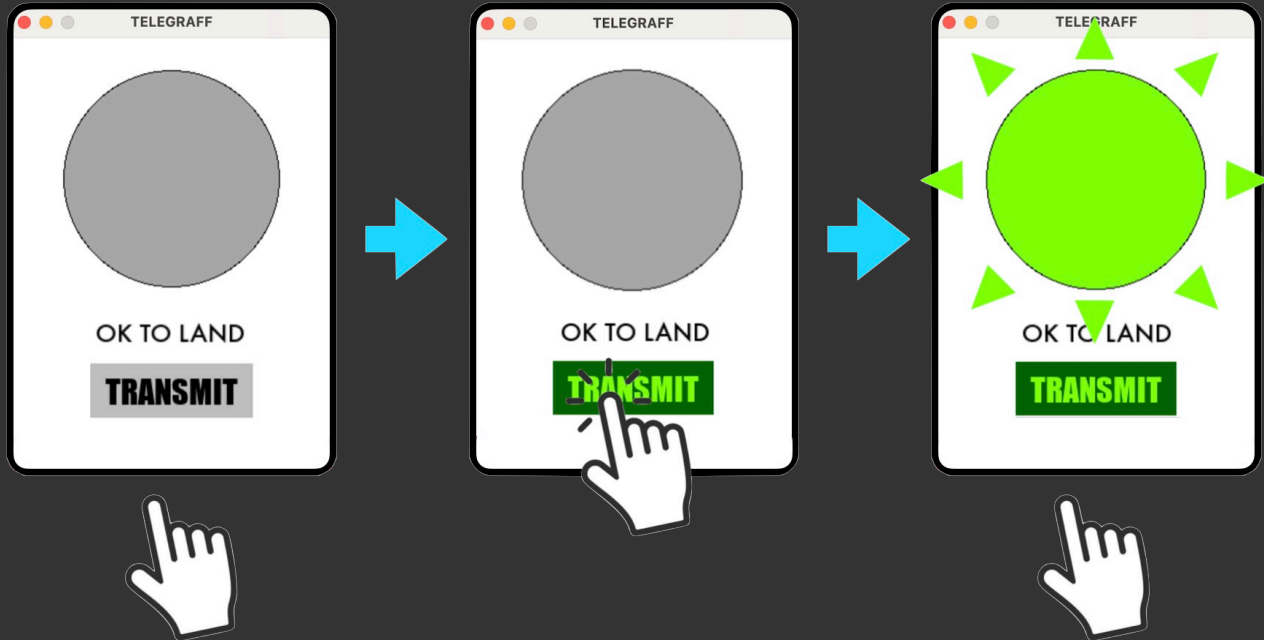


timer
events

CSCI
134

it'd be convenient if the app could send
morse code for a message typed by the user



suppose we wrote a function that computes the morse code "rhythm" of a message

A ··	N ··
B ····	O ···
C ····	P ····
D ···	Q ····
E ·	R ···
F ····	S ···
G ···	T ·
H ····	U ··
I ··	V ····
J ····	W ···
K ···	X ····
L ····	Y ····
M ··	Z ····

```
In [1]: from morse import compute_rhythm
```

```
In [2]: compute_rhythm("ART")
```

```
Out[2]: [1, 0, 1, 1, 1, 0, 0, 0, 1, 0, 1, 1, 1, 0, 1, 0, 0, 0, 1, 1, 1, 0, 0, 0]
```

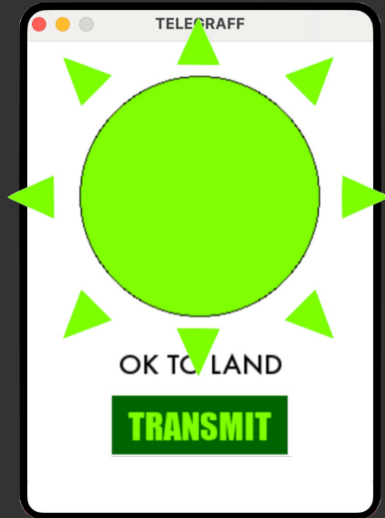


```
In [1]: from morse import compute_rhythm
```

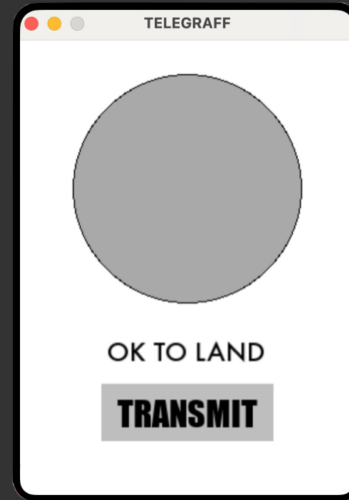
```
In [2]: compute_rhythm("ART")
```

```
Out[2]: [1, 0, 1, 1, 1, 0, 0, 0, 1, 0, 1, 1, 1, 0, 1, 0, 0, 0, 1, 1, 1, 0, 0, 0]
```

when it's a **1**, we should
signal



when it's a **0**, we should
not signal



this code causes the light to flash
on and off every 0.5 seconds

keeps track of
the current state →
of the lightbulb

```
class ProgramState:
    def __init__(self):
        self.is_transmitting = False

state = ProgramState()

def periodic_update():
    if state.is_transmitting:
        state.is_transmitting = False
        lightbulb.change_fill_color("darkgray")
    else:
        state.is_transmitting = True
        lightbulb.change_fill_color("chartreuse")

window.call_every_k_milliseconds(500, periodic_update)
```

this code causes the light to flash
on and off every 0.5 seconds

asks the window
to call the
`periodic_update`
function every
500 milliseconds →

```
class ProgramState:
    def __init__(self):
        self.is_transmitting = False

state = ProgramState()

def periodic_update():
    if state.is_transmitting:
        state.is_transmitting = False
        lightbulb.change_fill_color("darkgray")
    else:
        state.is_transmitting = True
        lightbulb.change_fill_color("chartreuse")

window.call_every_k_milliseconds(500, periodic_update)
```

this code causes the light to flash
on and off every 0.5 seconds

this function
flips the bulb →
on and off

```
class ProgramState:
    def __init__(self):
        self.is_transmitting = False

state = ProgramState()

def periodic_update():
    if state.is_transmitting:
        state.is_transmitting = False
        lightbulb.change_fill_color("darkgray")
    else:
        state.is_transmitting = True
        lightbulb.change_fill_color("chartreuse")

window.call_every_k_milliseconds(500, periodic_update)
```

this is an example of a "timer event"

```
window.call_every_k_milliseconds(500, periodic_update)
```



starts the timer

0

500

1000

1500

2000

2500ms



```
def periodic_update():  
    if state.is_transmitting:  
        state.is_transmitting = False  
        lightbulb.change_fill_color("darkgray")  
    else:  
        state.is_transmitting = True  
        lightbulb.change_fill_color("chartreuse")
```

changes the state

similarly, we can use a timer event to follow the morse code rhythm

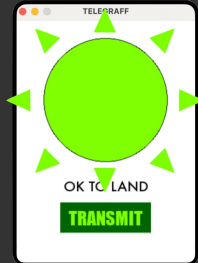
```
In [1]: from morse import compute_rhythm
```

```
In [2]: compute_rhythm("ART")
```

```
Out[2]: [1, 0, 1, 1, 1, 1, 0, 0, 0, 1, 0, 1, 1, 1, 1, 0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0]
```

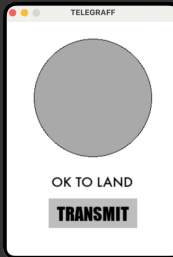
when it's a **1**, we should

signal



when it's a **0**, we should

not signal



```
class ProgramState:
```

```
    def __init__(self):
```

```
        self.is_transmitting = False
```

```
        self.message = ""
```

```
        self.rhythm = None
```

```
        self.rhythm_index = 0
```

```
state = ProgramState()
```

```
def periodic_update():
```

```
    if state.rhythm != None:
```

```
        if state.rhythm_index + 1 < len(state.rhythm):
```

```
            state.rhythm_index += 1
```

```
            if state.rhythm[state.rhythm_index] == 1:
```

```
                color = "chartreuse"
```

```
            else:
```

```
                color = "darkgray"
```

```
            lightbulb.change_fill_color(color)
```

```
window.call_every_k_milliseconds(120, periodic_update)
```